Huddle — PRD (v1)

Oct 11, 2026 · @Her

Overview

Huddle is a group trip planner: a web app where a group of friends shares one trip page, logs who paid for what, sees who owes whom in the fewest payments, and votes on decisions.

- **Problem:** group trips get planned across WhatsApp threads, and costs get tracked in notes and bank transfers. Nobody knows the final balance, and decisions like dates or trains stall in chat.
- **Goal:** one shared place per trip for money and decisions.
- **Users:** small friend groups (roughly 3–10 people) on a trip, e.g. a weekend hike by train.
- **Why it's worth building:** full-stack (auth, Postgres, access rules), multi-user, deployed with a live link, and one real algorithm (settle up). It's written entirely by me, not from a tutorial.

Scope

v1 covers accounts, trips with invite links, expenses with settle up, and polls; everything else waits.

In v1

- Sign up, log in, log out (email and password)
- Create a trip, invite friends with a link, see members
- Add, edit and delete expenses; split equally among the people you select
- Balances per person and a settle-up plan with the fewest payments
- Mark a suggested payment as paid
- Polls: create, vote, see results, close

Later (not v1)

- Shared day-by-day itinerary
- Packing / to-do list
- Custom-amount and percentage splits
- Multiple currencies (v1 is GBP only)
- Live updates without refreshing (Supabase Realtime)

- Receipt photos, notifications, Google sign-in
- Visual design polish (v1 uses a plain, clean UI)
- Leaving and removing members: a member can leave once their balance is £0.00; the organiser can remove a member, who must confirm (removed automatically after 7 days if settled)

User stories

Area	As a…	I want to…	So that…
Auth	new user	sign up with email and a display name	my friends see who I am
Trips	organiser	create a trip with a name and dates	we have one place for it
Trips	organiser	copy an invite link	friends can join without me adding them
Trips	friend	open an invite link and join after logging in	I land straight on the trip
Expenses	member	add an expense: what, amount, who paid, who's included	the cost is shared fairly
Expenses	member	leave out people who weren't there	they don't pay for a dinner they missed
Expenses	member	edit or delete an expense I added	mistakes can be fixed
Settle up	member	see what I owe or am owed	I know where I stand
Settle up	member	see the fewest payments that clear all debts	we settle with as few transfers as possible
Settle up	member	mark a payment as paid	balances update
Polls	member	create a poll with 2–6 options	the group can decide on dates, trains or places
Polls	member	vote, and change my vote while it's open	I can change my mind
Polls	poll creator	close the poll	the result is final

Requirements and rules

The money rules live in pure, tested TypeScript functions, separate from the UI, so the maths can be tested on its own (same approach as runFlex).

Accounts

- Email + password via Supabase Auth; a display name is required at sign-up.
- Logged-out users only see sign up / log in; an invite link they open is remembered and used after login.

Trips and invites

- A trip has a name, optional start and end dates, and an organiser (its creator).
- Each trip has one invite code; the link is `/join/<code>`. The organiser can reset it, which stops the old link working.
- Opening a valid link while logged in adds you as a member (no duplicates) and opens the trip.
- Only members can see a trip or anything in it.

Expenses

- Fields: description, amount, paid by (one member), included members (at least one), date.
- Amounts are stored as whole pence (integers), never floats, so totals always add up.
- Equal split: each included person's share is amount ÷ number included, rounded down to the penny. Leftover pence go one each to included members in a fixed order (by when they joined), so shares always sum exactly to the amount. Planned change (after the simple version is working and tested): if the payer is included, the payer takes the leftover pence instead, so the extra penny doesn't always fall on the same person; if the payer isn't included, it falls back to join order.
- The payer may or may not be included (e.g. paying for others' tickets).
- Only the person who added an expense, or the organiser, can edit or delete it.

Balances and settle up

- Each member's balance = what they paid for expenses − their shares + payments they've sent − payments they've received. Positive means they're owed money; balances across a trip always sum to 0.

- Settle-up plan (greedy): repeatedly match the person owed the most with the person who owes the most, and have the debtor pay the smaller of the two amounts. Repeat until everyone is at 0.
- This needs at most (members − 1) payments. Finding the true minimum is a much harder problem (NP-hard), so greedy is a deliberate trade-off, which makes a good interview talking point.
- "Mark as paid" records a payment (from, to, amount). The balances and plan then recalculate.

Polls

- A poll has a question and 2–6 options; single choice, one vote per member.
- Members can change their vote while the poll is open; results show the count per option and who voted for what.
- Only the creator can close a poll; closed polls show the result and accept no votes.

Screens

v1 needs six screens plus two tabs on the trip page; wireframes will be added here later.

Screen	Route	Shows	Main actions
Sign up / Log in	`/login`, `/signup`	Email, password, display name	Create account, log in
My trips	`/trips`	Trips I'm in, with dates and member count	New trip, open trip
New trip	`/trips/new`	Name, start and end dates	Create
Join trip	`/join/:code`	Trip name, organiser	Join
Trip page	`/trips/:id`	Members, invite link, tabs: Expenses · Balances · Polls	Copy link, switch tab
Add / edit expense	`/trips/:id/expenses/new`	Description, amount, paid by, who's included (checkboxes, all ticked by default), date	Save, delete
Balances	tab on trip page	Each member's balance; settle-up plan ("Sam pays Alex £12.50")	Mark as paid

Screen	Route	Shows	Main actions
Polls	tab on trip page	Open and closed polls with counts and voters	New poll, vote, close

Mobile-first layout, since people will add expenses on their phones during the trip.

Data model

Nine Postgres tables in Supabase; balances are calculated, not stored, so they can never go out of sync.

Table	Key columns	Notes
`profiles`	`id` (= auth user id), `display_name`	One row per user, created at sign-up
`trips`	`id`, `name`, `start_date`, `end_date`, `organiser_id`, `invite_code` (unique)	
`trip_members`	`trip_id`, `user_id`, `joined_at`	Primary key (`trip_id`, `user_id`) stops duplicate joins
`expenses`	`id`, `trip_id`, `description`, `amount_pence` (int), `paid_by`, `created_by`, `spent_on`	
`expense_participants`	`expense_id`, `user_id`, `share_pence` (int)	Shares written when the expense is saved; they sum to `amount_pence`
`payments`	`id`, `trip_id`, `from_user`, `to_user`, `amount_pence`, `created_at`	Created by "Mark as paid"
`polls`	`id`, `trip_id`, `question`, `created_by`, `closed_at` (null = open)	
`poll_options` + `poll_votes`	options: `id`, `poll_id`, `label` · votes: `poll_id`, `user_id`, `option_id`	Primary key (`poll_id`, `user_id`) = one vote each

Row-level security (RLS): every table is locked so you can only read or write rows for trips you're a member of. Joining goes through a small database function that checks the invite code, because a non-member can't otherwise read the trip. Edit and delete rules (creator or organiser) are enforced in RLS too, not just hidden in the UI.

Tech stack and testing

React + TypeScript on the front end, Supabase for the database, auth and access rules, deployed on Vercel.

Part	Choice	Why
Front end	React + TypeScript + Vite, React Router	Builds on what I already know from Roomify
Styling	Tailwind	Fast plain UI now, easy to polish later
Backend	Supabase: Postgres, Auth, RLS, one database function for joining	No separate server to run; access rules live in the database
Money logic	Pure TS module (`src/lib/money.ts`): `splitEqually` , `computeBalances` , `settleUp`	Testable without a database or UI
Tests	Vitest	Same test-first habit as pytest in runFlex
Hosting	Vercel (front end) + Supabase free tier	Free, and gives a public live link
Code	New GitHub repo, `.env` with Supabase keys never committed	

Testing plan

- Unit tests for the money module first: splits that don't divide evenly (£10 between 3 people), payer not included, balances summing to 0, settle-up producing at most (n − 1) payments, and everyone already settled.
- Manual test with two browser accounts for invites and RLS: a non-member must not be able to see a trip, even by guessing its URL.

Decisions made during the build

These were decided while building and override anything above that says otherwise.

Trips and invites (step 4)

- Opening an invite link shows a "Join [trip]?" page; joining takes one click rather than happening automatically.
- Any member can see and share the invite link; only the organiser can reset it (with a confirmation).
- "Trip not found" and "not a member" show the same message, so the app never reveals whether a trip exists.
- My trips is sorted newest created first; trip dates are optional, and an end date before the start date is rejected.
- After logging in or signing up, you return to the page you were trying to open (e.g. an invite link).

Expenses (step 5)

- An expense and its shares are saved by one database function (`save_expense`), so they run as a single transaction: all or nothing.
- Shares are calculated once, when saving, with `splitEqually` (people in join order). The stored shares are the single source of truth: `computeBalances` reads them rather than re-splitting, so a future change to the leftover-penny rule doesn't alter past expenses.
- The database rejects shares that don't add up to the expense amount.
- Amounts are typed in pounds and converted to pence as text (at most 2 decimal places), never via floating-point maths.
- Amounts over £1,000 ask "is that right?" but are allowed; there is no business limit on how big an expense can be. The only cap is technical: £20,000,000, just under what the database's integer column can hold, so anything bigger gets a friendly message instead of a database error.
- Editing uses a separate page that reuses the add form; deleting asks for confirmation.
- The expenses list is sorted newest first by date spent, and each row shows the amount, who paid and your share.

Balances and settle up (step 6)

- Balances and the settle-up plan are calculated in the browser with the tested `computeBalances` and `settleUp`, from the stored expense shares and recorded payments. Members are always in join order, so the same balances always give the same plan.
- The plan is recalculated live: a new expense can change who pays whom. Payments already recorded still count.

- Only the two people in a suggested payment see its "Mark as paid" button; marking it paid asks for confirmation and records the full suggested amount.
- Either the sender or the receiver can record a payment; it counts immediately (no receiver confirmation in v1).
- A payment history is shown, and the sender or receiver can undo a payment (with confirmation).
- The Balances tab shows: your personal summary, everyone's balance (yours highlighted), the settle-up plan (your payments first, "Everyone's settled up!" when empty), then payment history.

Polls (step 7)

- A poll and its options are created by one database function (`create_poll`), so they save as a single transaction. It checks there are 2–6 options, none blank and no duplicates. Options keep the order they were typed in.
- The creator or the organiser can close a poll. Closed polls are final: they can't be reopened or edited, and they accept no votes.
- Polls can't be edited after creation; the creator or organiser can delete one.
- You can change your vote while a poll is open, but not remove it.
- Results (counts and who voted for what) are always visible, even before you vote.
- When a poll closes on a tie, it shows "Tie between X and Y" with no automatic tie-breaker.
- The Polls tab lists open polls first, then closed, newest first within each. "+ New poll" opens a separate page, like Add expense.

Later (noted during the build)

- Partial payments: record a custom amount instead of the full suggested one.
- Receiver confirmation: the sender marks "I paid", and it only counts once the receiver confirms "I got it".
- More specific feedback in forms (already started with the amount error messages).
- Poll deadlines that close a poll automatically.
- Multiple-choice polls (vote for more than one option).
- Date-picker options for "which dates?" polls.
- Trip cover photos (v1.1, after deploying): any member can upload a cover photo (Supabase Storage, members-only access, size and type limits). It shows as a short banner on each My trips card and a larger banner at the top of the trip page; trips without a photo get a fallback cover in the accent tint with the trip's first letter.

Before launch

- [ ] Turn "Confirm email" back on in Supabase (it's off for development).
- [ ] Redirect logged-in users away from the log in and sign up pages.

Build order and definition of done

Build the money logic and data first, a plain UI on top, and visual design last.

1. Money module + Vitest tests (no UI, no database)
2. Supabase project: tables, RLS policies, join function
3. Auth: sign up, log in, log out, profiles
4. Trips: create, list, trip page, invite link and join flow
5. Expenses: add, edit, delete, list
6. Balances tab and settle-up plan, "Mark as paid"
7. Polls: create, vote, close
8. Wireframes, then visual design pass
9. Deploy to Vercel, README with screenshots and live link

v1 is done when: two real accounts can join one trip by link, add expenses with partial splits, see a correct settle-up plan, mark it paid, and run a poll, all on the live site, with the money tests passing.